

Generative AI - Assignment 1

Submission Sheet

Name	Darrsheni Sapovadia
Student ID	27PGAI0063
Submitted on	14 September 2026
Parts completed	Part 1 (45 marks) and Part 2 (55 marks)

GitHub repository

<https://github.com/darrshenisapovadia/genai>

The repository is public and can be cloned and run as it is. Both notebooks are committed with their outputs still in them, so the results can be read straight off GitHub without running anything.

What is in the repository

File	What it is
notebooks/Part1_News_Analysis.ipynb	Part 1, run end to end with outputs saved
notebooks/Part2_Job_Postings_Analysis.ipynb	Part 2, run end to end with outputs saved
data/	Both datasets, committed so the notebooks run after a plain clone
outputs/	The two finished dataframes written out as CSV
.env.example	Copy to .env and add your own Groq key
requirements.txt	The handful of packages needed

How to run it

1. git clone <https://github.com/darrshenisapovadia/genai>
2. pip install -r requirements.txt
3. Copy .env.example to .env and paste in a Groq API key
4. Open the notebooks folder in Jupyter and run all cells

The notebooks use relative paths, so they need to be run from inside the notebooks folder.

Part 1: news articles

First 30 articles of the BBC News Archive. Three LangChain chains, one for each task: a few-shot classifier that returns one of the five categories, a summariser limited to two or three sentences, and an entity extractor that returns JSON so the people, organisations and places come back in separate lists. The results go back onto the dataframe as Detected_Topic, Summary and Key_Entities, and the notebook finishes with the full merged dataframe.

The dataset ships with its own category labels, so I checked the predictions against them. The model matched 29 of the 30. The file is sorted by category though, so those first 30 are all business articles and that check on its own is a weak one. I took a second sample of ten articles, two from each of the five categories, and it got 10 of those right.

Earlier on it was reading business stories about trade rules or fuel prices as Politics, which is fair enough but not how the dataset labels them. Adding a line to the prompt saying that anything to do with the economy counts as Business, even when a government is involved, cleared nearly all of those up. The one it still gets wrong is an article about Ask Jeeves and online advertising, which it files under Tech rather than Business. Given the article is largely about a search engine, that is a defensible call.

Part 2: job postings

First 25 postings. One chain classifies the role into a broad domain, and a second pulls the skills, education level and experience out of the description in a single pass. I used one combined extraction prompt rather than three separate ones, which cuts the API calls to a third and means the model reads the description once instead of three times. The new columns are Predicted_Category, Required_Skills, Education_Required and Experience_Required.

A lot of these adverts are thin. 10 of the 25 say nothing at all about education, so the prompt tells the model to write Not specified rather than guess at a degree that was never mentioned.

Things I had to work around

The brief suggests llama-3.1-8b-instant on Groq. That model has since been retired and the API now returns a 404 for it, so I used openai/gpt-oss-120b instead. It is on the same free tier and handles this kind of work well. Everything runs at temperature 0 so the results repeat.

The one that cost me the most time: this model reasons before it answers, and the reasoning counts towards max_tokens. On the longer articles it used the whole budget thinking and returned an empty string, so my Key_Entities column came out blank for most rows without anything actually throwing an error. Setting reasoning_effort to low fixed it and made the run quicker. I also changed my retry helper to raise on the last attempt instead of returning an empty string, since that is what let the problem go unnoticed in the first place.

The free tier also caps you at 8,000 tokens a minute, and Part 1 makes three calls per article. Both notebooks pause between rows and retry if they get rate limited, so a full run takes a few minutes.